

END-OF-COURSE PROJECT · MILESTONE 1

Project Problem Statement

Submit by the end of Week 3 [23 August 23:59 hrs] · ~1 page · individual · formative (not graded) — you'll get written feedback.
This seeds your End-of-Course Project (it maps to Project Rubric criteria 1 & 2).

This submission is NOT graded. However, on time non-submission counts towards incomplete End of course project.

Name: WANG KENAN Section (A / B / C): B Date: 2026/8/21

1 · Working title

A short, specific name for your project.

Customer-ticket triage — classify banking service tickets for back-office duty managers

2 · The problem, and why it matters

Who has this problem, and why is it worth solving? 2–3 sentences. (Rubric 1)

Single core problem(one-sentence): Unstructured retail-bank customer complaint tickets are currently triaged purely by keyword-filter rules, which miss many high-priority complaints and force human staff to read every incoming ticket manually. The closest existing tool is rule-based keyword ticket filtering widely used in bank service desks; it cannot capture rephrased customer grievances that do not contain pre-defined trigger words. At my simulated workload, manual review costs roughly 2.5 human working-hours for every 100 tickets, and urgent complaints face delayed response times.

Out-of-scope: This system will not resolve customer complaints directly; it only produces triage flags for human operators.

3 · Who it's for, and the domain

Your user / audience and the industry (e.g. banking, healthcare, retail, your employer).

Primary user: back-office bank duty shift supervisor.

One-line persona: Sarah, a back-office shift supervisor finishing her afternoon desktop shift, processing incoming customer tickets; she cannot interpret raw model reasoning text and only needs a clear triage flag for each case. Domain: Retail banking customer service. Once my system works, Sarah will no longer need to read every full customer ticket; she only conducts manual review for tickets marked for escalation.

4 · Why AI — and which kind? Remember, from class 1 workshops, no one size fits all

Is AI the right tool here (vs a rule or lookup)? If yes, which type — narrow ML, a foundation model, RAG, an agent? (ILO 1)

A pure keyword-matching rule set acts as my **non-AI baseline**. Keyword rules fail for re-phrased complaints, so AI delivers tangible incremental value for this task.

I decide to use rented foundation-model structured prompting (Class3-C2). I will not adopt an agent; multi-step tool-call loops add latency and token cost and are unnecessary for this single-step classification. Fine-tuning is rejected because I lack thousands of fresh task-specific labelled samples. RAG is unnecessary here since no private enterprise documents need retrieval. I will keep simple regex sanity-check rules alongside LLM outputs as hybrid guardrails. My LLM acts as classification component and also serves as LLM-as-judge for later evaluation; I keep LLM away from final safety-critical business decisions, which remain for human staff. My prompting ladder stays at rung 1-4 (clear instruction, few-shot examples, structured JSON output). I will not enable self-consistency to avoid multiplied token expense.

5 · Proposed approach — build vs buy or both!

What will you build (low-code and/or code)? Which layers will you own vs rent, and why? (Rubric 2, ILO 3)

I analyse each layer of the Class2 modern AI stack and mark **own / rent with explicit reasoning**:

1. Compute: **Rent**. I will use public LLM API cloud runtime; self-hosting foundation-model hardware brings prohibitive up-front capital cost for this student project.
2. Model: **Rent**. I rent GPT-4o-mini closed-source API; I will not fine-tune or retrain base model weights.
3. Data & Retrieval: **Own**. I handle dataset filtering, train-test splitting and leakage detection myself; no retrieval component is required for this task.
4. Orchestration: **Own**. I build prompt templates, few-shot examples, schema validation and pre-processing logic in Python; this contains my project-unique business logic.

5. Serving: **Rent**. I use notebook-style lightweight runtime; building production-grade deployment infrastructure falls outside project scope.
6. Evaluation & Observability: **Own**. I build full L1 assertion and LLM-as-judge L2 evaluation harness, which represents my core validation work.

Low-code consideration: I first considered n8n low-code workflow builder to invoke model APIs. n8n cannot support my custom structured-output assertion evaluation logic. Therefore I choose to implement everything in Python code.

Cost-to-serve estimation(token economics): Each request carries ~320 input tokens(system prompt + few-shot samples), ~70 output tokens(JSON result). Self-consistency is disabled(n=1). Per-call estimated inference cost is low; scaling to 1 000 000 tickets will multiply total token expenditure linearly.

6 · Data – this packs the intelligence !

What data do you need, where from, and any access / privacy / residency constraints?

Dataset: Kaggle Bank Customer Complaint Dataset

Source link: public Kaggle bank-customer-complaints dataset

Row count: 18 000+ anonymised labelled banking customer tickets

Licence: CC-0 public academic-use licence, no redistribution restrictions.

I have downloaded and manually inspected sample rows to confirm dataset content matches my ticket-triage task definition.

Processing: I split out 220 held-out test samples for evaluation; remaining data supplies few-shot example candidates.

PDPA consideration: All records are fully anonymised with zero PII, so personal-data consent requirements do not apply.

Leakage mitigation: I strictly ensure none of my held-out test samples appear inside few-shot prompt examples. I will compare model performance before and after leakage removal.

7 · Success metric & how you'll evaluate it

One metric that ties to real value, and one check (an eval) you could actually run. (evals thread)

One value-tied primary metric: **Macro F1-score for the ESCALATE class.**

- Baseline F1(keyword non-AI baseline): 0.64
- My project target F1-score: ≥ 0.82

Evaluation design:

1. L1 automatic assertions: Every output must conform strictly to pre-defined JSON schema, returning exactly ESCALATE / MONITOR / ABSTAIN. Malformed JSON counts as failed test case.
2. Held-out evaluation set of 220 unseen tickets, untouched during prompt tuning.
3. System supports explicit abstain mechanism: When confidence is low, it outputs ABSTAIN and routes case directly to human reviewer. I will measure two figures: abstain frequency, and the proportion of abstained samples that the model would otherwise mis-classify.
4. I deploy an LLM-as-judge for L2 evaluation. Since the judge model itself is a system component, I will calibrate its performance against my hand-labelled subset to measure its own precision and recall. I avoid tuning prompts against test-set data.

8 · Risks, limitations & responsible use

What could go wrong (incl. silent failure), and who might it affect?

Each risk paired directly with mitigation; disclaimer alone is not accepted as mitigation

1. **Silent-failure risk:** Model confidently produces wrong triage label for nuanced customer complaints, with no obvious signal of error. Mitigation: Mandatory human-in-the-loop workflow; every model output still undergoes human supervisor review; model output is advisory only and never constitutes a final decision.
2. Risk: Poor performance on rare complaint categories not covered within few-shot examples. Mitigation: Leverage the ABSTAIN flag; low-confidence inputs are automatically diverted to human staff.

3. Limitation: Performance will degrade for complaint types absent in the original Kaggle dataset. Mitigation: Document limitation clearly in demonstration material.
4. Intended use: Internal bank-back-office triage assistance only. Explicit non-use: Cannot be deployed to make final customer-facing decisions. Reference governance frameworks: Singapore IMDA Model AI Governance Framework, OWASP Top-10 for LLM Applications (2025).

9 · Smallest first version (optional)

The tiniest end-to-end slice you could build first.

My minimal complete end-to-end slice: One raw customer-ticket-text input → single LLM API call → strictly-structured JSON output containing triage label (ESCALATE / MONITOR / ABSTAIN). No additional features: No batch processing, no graphical UI, no persistent logging. I describe functionality slice; I do **not** write deadlines as a version definition.

Working-check sentence: When I feed real-world sample complaint text as input, I obtain valid schema-compliant JSON output. If this check fails, any features built on top are meaningless. Realistic implementation window is one week after A1 deadline(30 August 2026). All later features(batch run, full evaluation harness) will extend from this already-working minimal slice.

How this is used

This milestone is not graded — it's to give you fast, written feedback while there's still time to change direction. A strong problem statement makes criteria 1 (problem & significance) and 2 (business & technical trade-offs) and helps you start on a strong footing. Bring questions during class breakouts or email the course account or WA text me

INSTRUCTOR FEEDBACK

For marker use — formative comments on the problem & significance (criterion 1) and the business/technical trade-offs (criterion 2), plus one thing to strengthen before the final project.